

KOGI

Independent Worker Operating System

Work Management System

System Design Document

v1.0 · Kogi Platform · March 2026

1. Overview

The Kogi Work Management System (KOGI-WMS) is the planning, execution, and coordination layer of the Kogi Independent Worker Operating System. It provides independent workers, collectives, cooperatives, and autonomous organizations with a unified workspace for managing every unit of work — from high-level portfolio themes down to individual tasks — together with the boards, timelines, analytics, resource tools, and content systems needed to deliver outcomes.

KOGI-WMS is designed around the Work Breakdown Structure (WBS) model, ensuring that all work is traceable from strategic vision to tactical execution. Every work item carries rich metadata, governance policies, CRDT versioning, and analytics hooks consistent with the wider Kogi Portfolio System.

Module	Purpose
Workspace	Central hub: dashboard, boards, timelines, analytics, resource management, content
Work Breakdown Structure	Hierarchical decomposition: Theme → Initiative → Epic → Story → Task
Work Studio	Requirements management and work design tooling
Governance	Policies, approvals, access control, audit trails
Analytics & Optimization	Forecasting, KPIs, OKRs, telemetry, performance dashboards
Resource Management	Budgeting, allocation, delegation, reporting
Content Management	Files, documents, contracts, SOPs, policies, frameworks

2. Workspace

The Workspace is the primary interactive environment for work management on the Kogi platform. It aggregates all active programs, projects, and tasks into a configurable working surface with context-sensitive views and real-time Oba AI assistance.

2.1 Work Dashboard

The Work Dashboard provides a personalized, high-density overview of all active work across the user's portfolios, programs, and projects. It is the entry point into the Workspace module.

Widget	Description	Data Source
Active Items Summary	Count of open themes, initiatives, epics, stories, and tasks with status distribution	WBS Engine
Priority Queue	Top 5–10 highest-priority in-progress items sorted by urgency and dependency	OptimizationEngine
Velocity Tracker	Story points / items completed per sprint or timebox with trend line	AnalyticsEngine
Blocker Alerts	Real-time flags on blocked stories and unresolved dependencies	WBS Engine + Oba
Upcoming Milestones	Next 30-day milestone and goal timeline	BookingEngine + Timeline
Team Capacity	Available vs. allocated capacity by member and role	AllocationEngine
OKR Progress	Key results progress meters per active Objective	GoalEngine
Quick Actions	Create story, log task, open board, open backlog	Workspace Router

2.2 Work Backlogs & Backlog Management

The backlog is the authoritative ordered list of work items for a program, project, or team. KOGI-WMS supports multiple backlog scopes operating simultaneously.

Backlog Type	Scope & Description
Program Backlog	All epics and stories across all projects within a program; managed by Program Lead
Project Backlog	All stories and tasks scoped to a single project; managed by Project Owner

Backlog Type	Scope & Description
Sprint / Timebox Backlog	Stories committed to the current sprint or PI increment
Team Backlog	Personal or team-specific task queue, filtered from the project backlog
Triage Queue	Ungroomed submissions (bugs, requests, ideas) awaiting refinement and prioritization
Archive	Closed, cancelled, or deferred items retained for historical analysis

Backlog Management Operations

- Prioritization — drag-and-drop ordering; bulk re-rank by score; MoSCoW and WSJF scoring
- Refinement — story splitting, acceptance criteria authoring, estimation (story points, T-shirt, hours)
- Grooming sessions — structured backlog review workflow with agenda, notes, and outcome capture
- Dependency mapping — visual dependency graph overlaid on backlog; blocker detection
- Capacity planning — sprint loading against team velocity with over-allocation warnings
- Bulk operations — multi-select: assign, label, move, archive, delete, change status

2.3 Work Governance

Work Governance enforces access control, approval workflows, and audit accountability across all work items. Every WBS item inherits policies from its parent component unless explicitly overridden.

Governance Feature	Description
Role-Based Access Control	Owner, Editor, Contributor, Viewer, Reviewer roles per component; inherited or overridden
Approval Workflows	Configurable gate: Autonomous Steward Review Governance Vote; required on status transitions
Change Proposals	Formal proposals for scope changes to Epics or Programs; voted on by stakeholders
Audit Trail	Full EventLog of all mutations (create, edit, status change, reassign) with actor, timestamp, delta
Policy Enforcement	PolicySet attached to component governs who can do what; enforced by WBS Engine at write time
Conflict Resolution	Concurrent CRDT edits merged automatically; unresolvable conflicts surfaced to owners as review items
Governance Reports	Auto-generated governance activity summaries for oversight and compliance needs

2.4 Work Content Management System

The Work CMS manages all structured documents and knowledge assets associated with work items, programs, and projects. Every document is a Portfolio Artifact with full versioning, access control, and lifecycle management.

Document Type	Description
Files	Binary and media attachments: images, exports, spreadsheets, raw assets linked to work items
Documents	Rich-text documents: notes, meeting summaries, project briefs, reports, planning docs
Contracts	Engagement contracts auto-populated from booking or work item details; e-signature workflow
Agreements	NDAs, IP assignments, service agreements, MoUs; templated and version-controlled
SOPs	Standard Operating Procedures: repeatable process definitions tied to project types or phases
Policies	Access control, governance, contribution, and data policies governing work items
Procedures	Step-by-step operational procedures; linked to tasks as execution checklists
Frameworks	Strategic and process frameworks: Agile, SAFe, OKR, PMBOK adaptations; reusable across programs
Models	Business models, data models, process models, decision trees stored as versioned documents

CMS Operations

- **Templates** — pre-built document templates for contracts, SOPs, briefs, retros, sprint reviews
- **Version history** — full revision history with diff view and restore
- **Access control** — per-document read/write/share permissions inherited from parent work item
- **Search & discovery** — full-text indexed across all document types; tag and label filtering
- **Linking** — bi-directional links between documents and WBS items (stories, epics, programs)

2.5 Work Boards

Work Boards provide kanban-style and grid-style visual management of work items. Each board is a configurable view of a backlog, filtered and grouped by any combination of fields.

Board Type	Description
Kanban Board	Swimlane columns representing status stages; drag-and-drop item progression
Agile Board	Sprint-scoped board with columns: To Do → In Progress → In Review → Done
Resource Board	Work items grouped by assignee; workload heatmap overlay
Priority Board	Items sorted by priority score; MoSCoW quadrant view available
Program Board	SAFe-style PI planning board: team rows × iteration columns; dependency arcs
Custom Board	User-defined columns, groupings, and filters saved as named views

Board Configuration Options

- Column definitions — map to story status values; add WIP limits per column
- Swimlanes — group rows by assignee, label, epic, priority, or custom field
- Card display — choose which fields appear on card face: title, assignee, points, due date, labels
- Filters — live filter by type, label, assignee, sprint, tag, epic, or any custom field
- Board policies — enforce WIP limits, require approval on column transitions, auto-archive on close

2.6 Work Timelines

Work Timelines provide time-axis visualization of work items across multiple time horizons. All timeline types share the same underlying work data; they differ in granularity and visual encoding.

Timeline Type	Granularity	Primary Use Case
Schedule	Day / Hour	Daily task and event scheduling; connects to booking and logistics calendars
Gantt	Day / Week	Dependency-aware project scheduling; critical path highlighting; resource bars
Calendar	Day / Week / Month	Event, milestone, and deadline view; integrated with kogi-bank invoices and booking
Roadmap	Month / Quarter / Year	Strategic initiative and program-level milestones; shareable externally
Program Increment (PI)	2-week sprints × 5 per PI	SAFe PI planning: objectives, sprints, and team commitments per increment
Sprint	1–4 weeks (configurable)	Active sprint scope with burndown; links to backlog and board
Custom Timebox	User-defined start / end + sub-periods	Bespoke planning cycles: weekly OKR reviews, quarterly cycles, campaign windows
Quarters	Q1–Q4	Fiscal and operational quarterly planning overlay on roadmap

Timebox Model

A Timebox is the base unit of time planning in KOGI-WMS. All schedule types (sprint, PI, quarter, custom) are Timebox instances with the following data shape:

Field	Description
timebox_id	UUID — unique identifier
name	Display name (e.g., 'Sprint 12', 'Q3 2026', 'PI 4')
timebox_type	Enum: Sprint PI Quarter Custom Duration
start_date / end_date	Inclusive date range
parent_timebox_id	Optional — links sprint to its parent PI
linked_program_id	Portfolio Program this timebox belongs to
committed_items	Vec<ComponentId> — WBS items committed to this timebox
capacity	Optional planned capacity in points or hours

Field	Description
status	Planning Active Review Closed

2.7 Work Analytics

Work Analytics provides real-time and historical visibility into delivery performance, team health, forecast accuracy, and strategic goal alignment. Analytics are computed by the AnalyticsEngine, TelemetryEngine, and OptimizationEngine, and surfaced through configurable dashboards.

Analytics Category	Metrics	Engine
Forecasting	Velocity-based sprint/release forecasting, Monte Carlo simulations, delivery date confidence intervals	OptimizationEngine
Performance	Throughput, cycle time, lead time, WIP levels, defect escape rate, rework ratio	AnalyticsEngine
Telemetry	Real-time activity feeds, item state change rates, contributor activity heatmaps	TelemetryEngine
Optimization	Bottleneck detection, flow efficiency, WIP-to-throughput ratio, queue aging	OptimizationEngine
Personalization	Individual performance summaries, personalized workload recommendations, learning curve tracking	PersonalizationEngine
KPIs	Configurable key performance indicators per program, project, team, or individual	AnalyticsEngine
OKRs	Objective tracking with key result progress meters, confidence scores, weekly check-ins	GoalEngine
Data Tracking	Custom event tracking on any WBS item field; exportable to CSV, JSON, or BI connector	DataStreamingEngine

Standard Reports

- Sprint Burndown — planned vs. completed points/items per day
- Cumulative Flow Diagram — WIP distribution across status columns over time
- Release Burnup — cumulative scope vs. completed work toward release target
- Team Velocity History — sprint-over-sprint velocity with moving average
- Cycle Time Scatter Plot — item-level cycle times with percentile bands
- Dependency Risk Report — blocked and at-risk items with dependency chain visualization
- OKR Progress Report — key result attainment rates with contributor attribution
- Capacity Utilization Report — allocated vs. available hours per person per period

2.8 Work Resource Management

Resource Management in KOGI-WMS covers the planning, allocation, delegation, and reporting of all resources required to execute work — including human capacity, budget, and tools.

Resource Feature	Description
Budgeting	Per-project and per-program budget envelopes; planned vs. actual cost tracking; kogi-bank integration for real expenditure data
Reporting	Financial and resource utilization reports; export to PDF or kogi-bank tax reports; scheduled delivery
Allocation	Assign workers, skills, and tools to epics and stories; capacity reservation with conflict detection by AllocationEngine
Delegation	Delegate task ownership and sub-task responsibility; delegation chain tracked with full audit trail
TODO Management	Personal and team TODO lists linked to WBS items; prioritized by due date and dependency; Oba surfaces overdue items

3. Work Studio

The Work Studio is the design-time environment within KOGI-WMS for structured requirements authoring, work design, and process engineering. It bridges the gap between strategic planning (portfolio and OKRs) and execution (epics and stories).

3.1 Requirements Management System

The Requirements Management System (RMS) provides a structured workflow for capturing, classifying, tracing, and validating requirements throughout the work lifecycle.

RMS Feature	Description
Requirement Capture	Structured forms for business, functional, and non-functional requirements; auto-linked to epics and stories
Requirement Types	Functional · Non-Functional · Business · Technical · Regulatory · Security · Performance
Traceability Matrix	Bi-directional trace from requirement → epic → story → task → test; gap detection
Acceptance Criteria	Gherkin (Given/When/Then) and free-form criteria authoring at story level; test linkage
Change Impact Analysis	Scope change to a requirement propagates impact warnings to all linked WBS items
Requirement Versioning	All requirement changes versioned; diff view; roll back to any prior version
Validation Workflow	Draft → Review → Approved → Baselined; approval gate enforced by governance policies
Export	Requirements export to CSV, DOCX, or structured JSON for external system handoff

3.2 Work Design Systems

Work Design Systems are reusable, composable frameworks for structuring programs and projects consistently across teams, collectives, and organizations on the Kogi platform.

Design System Asset	Description
Work Templates	Pre-configured WBS structures for common work types: software project, event production, content campaign, cooperative launch
Process Blueprints	Defined phase gates, workflow steps, and decision points for standard process types

Design System Asset	Description
Story Type Libraries	Curated story type taxonomies (see WBS section) per domain: product, creative, operations
Definition of Ready (DoR)	Checklist of conditions a story must meet before sprint commitment; enforced at board entry
Definition of Done (DoD)	Checklist of conditions required before a story is marked complete; enforced at column exit
Estimation Models	Story point scales, T-shirt sizing rubrics, affinity mapping guides; calibrated to team velocity history
Retrospective Formats	Structured retro formats: 4Ls, Start/Stop/Continue, Mad/Sad/Glad; outcomes captured as portfolio artifacts

4. Work Breakdown Structure (WBS)

The Work Breakdown Structure is the hierarchical decomposition model that organizes all work on the Kogi platform from strategic themes to atomic tasks. Every level is a typed Portfolio Component with full metadata, governance, analytics, and lifecycle support.

WBS Level	Definition
Theme	The highest strategic level — a broad area of investment or platform capability; spans multiple initiatives
Initiative	A coordinated set of epics advancing a Theme; maps to a platform program or a community initiative
Epic	A large body of work delivering a meaningful capability; decomposed into stories; fits within a PI or quarter
Story	The primary unit of delivery — a discrete, estimable, testable slice of value; fits within a sprint or timebox
Task	An atomic unit of execution within a story; assigned to a specific worker; completed in hours or a day

4.1 WBS Hierarchy

```

THEME
├── INITIATIVE
│   ├── EPIC
│   │   ├── STORY (+ STORY.DATA + STORY.TYPE)
│   │   │   └── TASK

```

4.2 Work Package

A Work Package is any addressable node in the WBS tree. All work packages share a common data envelope (story.data) and carry a type classification (story.type).

4.3 story.data — Work Item Data Model

Every WBS item from Epic to Task carries the following core data fields:

Field	Type	Description
unique_id	UUID	Platform-wide unique identifier; immutable once assigned

Field	Type	Description
name	String	Display name of the work item
owners	Vec<EntityId>	One or more users or orgs with ownership permissions
labels	Vec<String>	Free-form labels for grouping and filtering
categories	Vec<Enum>	Domain categories (Product, Engineering, Design, Operations, etc.)
classes	Vec<Enum>	Class of service: Standard Expedite Fixed-Date Intangible
types	StoryType	Functional classification — see story.type table
dependencies	Vec<ComponentId>	Items this work item depends on (must complete first)
dependents	Vec<ComponentId>	Items that depend on this work item
children	Vec<ComponentId>	Sub-items in the WBS hierarchy
parents	Vec<ComponentId>	Parent items in the WBS hierarchy
attachments	Vec<ArtifactId>	Linked files, documents, and CMS artifacts
fields	Map<String, Value>	Custom fields defined per project or program template
created_at	DateTime	ISO 8601 creation timestamp; immutable
updated_at	DateTime	ISO 8601 last update timestamp; auto-updated on any mutation
tags	Vec<String>	Hashtag-style discovery tags; indexed for search and analytics
status	Enum	Current lifecycle state: Backlog Ready In Progress In Review Done Archived
priority	Enum	Critical High Medium Low Unprioritized
estimate	Option<Estimate>	Story points, hours, or T-shirt size; set during refinement
policy_ids	Vec<PolicyId>	Governance policies governing this item
vector_clock	VectorClock	CRDT conflict resolution clock; auto-managed by platform

4.4 story.type — Work Item Type Taxonomy

Story types classify the functional nature of a work item, enabling filtering, routing, reporting, and workflow customization. A work item carries one primary type and optional secondary types.

Type	Category	Description
Feature	Delivery	New user-facing capability or product increment
Bug	Quality	Defect in existing behavior causing incorrect output
Testing	Quality	Test case authoring, test execution, or QA coverage work
Capability	Delivery	Technical or platform enabler delivering underlying functionality for multiple features
Issue	Operations	General operational issue or concern raised for resolution
Defect	Quality	Internal defect in design, code, or process; may not be user-visible
Enhancement	Delivery	Improvement to existing functionality rather than net-new feature
Innovation	Strategy	Exploratory work: prototype, spike, experiment, or innovation sprint item
Audit	Compliance	Scheduled review of process, security, governance, or compliance status
Enabler	Delivery	SAFe enabler: infrastructure, architecture, exploration, or compliance support work
Blocker	Operations	Item whose resolution is blocking other items; requires escalated attention
Use Case	Requirements	Functional scenario documenting actor-system interaction
Business Case	Strategy	Structured justification for a proposed initiative or investment
Requirement	Requirements	Formal requirement (functional, non-functional, business, or technical)
Documentation	Knowledge	Documentation creation or update: guide, runbook, SOP, API doc
Milestone	Planning	A significant event or decision point in a program or project
Goal	Planning	A desired outcome or target state; parent of key results or epics
Objective	Planning	OKR Objective — aspirational direction for a team or program
Outcome	Planning	Measurable result or impact produced by completing a set of work items
Mission	Strategy	Enduring purpose statement for a team, org, or program

Type	Category	Description
Vision	Strategy	Long-horizon aspiration describing the desired future state
Risk	Governance	Identified risk with likelihood, impact, and mitigation plan
Strategy	Strategy	Strategic direction or approach document; parent of initiatives
Tactic	Operations	Specific action plan executing a strategy; scoped to a timebox
Operation	Operations	Recurring operational work: maintenance, support, monitoring
Plan	Planning	Structured plan artifact: sprint plan, release plan, capacity plan
Report	Knowledge	Deliverable report: sprint review, status report, retrospective output
Release	Delivery	A versioned set of work items bundled for deployment to users
Deployment	Delivery	A specific deployment event: production push, rollout, environment promotion
Distribution	Delivery	Distribution of a release, asset, or resource to recipients or channels
Template	Knowledge	Reusable pattern or starting configuration for a work item or document type
Archive	Operations	Closed or deprecated items retained for reference; no active work

5. Engine Integrations

KOGI-WMS is powered by the Kogi Data Engine layer. The following engines provide the intelligence and automation backbone for the Work Management System:

Engine	Role in Work Management
AnalyticsEngine	Computes velocity, throughput, cycle time, lead time, and all standard delivery metrics
TelemetryEngine	Real-time event streaming for item state changes, contributor activity, and system health
OptimizationEngine	Forecasting models (Monte Carlo), bottleneck detection, flow efficiency optimization
PersonalizationEngine	Personal dashboard curation, individual workload recommendations, notification tuning
AllocationEngine	Resource allocation, capacity planning, conflict detection, delegation management
SearchEngine	Full-text search across WBS items, documents, requirements, and CMS artifacts
GraphEngine	Dependency graph computation, impact analysis, critical path detection
RiskEngine	Blocker risk scoring, dependency risk propagation, delivery confidence scoring
IncentiveEngine	Contribution attribution and incentive tracking for cooperative work and shared portfolios
GameEngine	Gamification mechanics: points, badges, streaks, and leaderboards for work completion
DataStreamingEngine	Real-time data export, BI connector feeds, event bus for third-party integrations
GoalEngine	OKR lifecycle management: objective creation, key result tracking, check-in scheduling
BookingEngine	Calendar and schedule integration; deadline coordination with booking and logistics systems
CollaborationEngine	Shared portfolio coordination, contribution attribution, concurrent edit conflict resolution

5.1 gRPC Engine Interface

All engines are accessible via the EngineGrpcServer on default port 9100. Work management components interact with engines through the following standard methods:

gRPC Method	Description
Control	Start, stop, configure, or reset engine subsystems
Status	Query engine health, active job counts, and last computation timestamp
Ingest	Push events, item mutations, and telemetry data to engine input stream
Snapshot	Request a point-in-time snapshot of computed engine state (e.g., current velocity, allocations)

Wire format: google.protobuf.Struct. gRPC reflection enabled by default; disable with KOGI_ENGINE_GRPC_REFLECTION=off.

6. Platform Integrations

KOGI-WMS is deeply integrated with adjacent Kogi platform systems. The following cross-system integrations are supported natively:

Kogi System	Integration Surface	Data Flow
kogi-bank	Budget envelopes, expense tracking, invoice generation, contractor payroll, grant disbursement tracking	Bi-directional: WMS reads/writes budget and expense data to kogi-bank ledgers
Portfolio System	All WBS items are Portfolio Components; full metadata, CRDT, governance, and analytics framework inherited	WMS items are first-class Portfolio objects; synchronized via GraphEngine
Community & Spaces	Shared portfolio programs, crowdresourced epics, initiative coordination across orgs and collectives	WMS items can be published to community spaces; contributions tracked via CollaborationEngine
Booking & Logistics	Schedules and deadlines synchronized; resource bookings linked to project timeline items	BookingEngine events create WMS timeline milestones and schedule blocks
Marketplace & Exchange	Work packages and gig tasks publishable to Marketplace; talent matching via MatchEngine	Open work items can be listed as gigs; applications flow into WMS assignment workflow
Developer / API	External systems can create, read, update, and close WBS items via REST and gRPC APIs	Full CRUD via kogi-api; webhook notifications on item state transitions

7. Glossary

Term	Definition
WBS	Work Breakdown Structure — hierarchical decomposition of work from Theme to Task
Work Package	Any addressable node in the WBS hierarchy carrying story.data and story.type
Timebox	A fixed time period bounding a set of committed work items (sprint, PI, quarter, custom)
Program Increment (PI)	A SAFe-style planning interval comprising 5 sprints; the primary planning cadence for large programs
Backlog	An ordered list of work items awaiting refinement, prioritization, or sprint commitment
Epic	A large body of work decomposed into stories, fitting within a quarter or PI
Story	The primary deliverable unit: estimable, testable, and completable within a sprint
Task	An atomic unit of execution within a story; typically completed within hours or a single day
Definition of Ready	Checklist of conditions a story must meet before being pulled into a sprint
Definition of Done	Checklist of conditions a story must satisfy before being marked complete
CRDT	Conflict-free Replicated Data Type — the distributed data structure enabling concurrent edits
GoalEngine	Kogi engine subsystem managing OKR lifecycle: objectives, key results, check-ins
AllocationEngine	Kogi engine subsystem managing resource allocation and capacity planning
CollaborationEngine	Kogi engine subsystem coordinating shared portfolios and contribution attribution
Oba	The Kogi AI assistant; surfaces recommendations, alerts, and optimization suggestions across all modules
EventLog	An append-only audit log attached to every Portfolio Component; records all mutations
Portfolio Component	The base data model for all items on the Kogi platform; WBS items are Portfolio Components

